

虚拟内存管理

操作系统为应用进程提供虚拟内存抽象。从应用进程的角度来看，每个进程有自己独立的虚拟内存空间，且无法直接访问其他进程的内存。代码片段 4.1 是一个简单的应用进程 `hello-vm`。该程序会调用三次 `printf` 向屏幕打印输出，后两次 `printf` 分别打印字符串变量 `str` 的地址和主函数 `main` 的地址。前者是数据地址（`0x411038`），后者是代码地址（`0x400614`），如输出记录 4.1 所示。这两个地址都是虚拟地址（Virtual Address）——应用进程在运行期间仅使用虚拟地址，即无论是数据地址还是指令地址均是虚拟地址[⊖]。然而，应用程序的数据和代码会被操作系统加载到物理内存中，访问物理内存实际上只能使用物理地址（Physical Address）。这就引出了两个值得思考的问题。

代码片段 4.1 应用进程 `hello-vm`

```
1 #include <stdio.h>
2
3 char str[] = "Hello VM!";
4
5 int main(void)
6 {
7     printf("%s\n", str);
8     printf("The (data) address of str is %p.\n", str);
9     printf("The (code) address of main is %p.\n", main);
10    return 0;
11 }
```

输出记录 4.1 `hello-vm` 的输出结果（运行于 AArch64 平台、Linux 操作系统）

```
[user@osbook ~] $ ./hello-vm
Hello VM!
The (data) address of str is 0x411038.
The (code) address of main is 0x400614.
```

⊖ 在一些嵌入式操作系统中，应用进程直接使用物理地址，这不在本书讨论范围内。

问题一：既然需要使用物理地址读写物理内存上的内容，操作系统为什么不直接让应用进程使用物理地址呢？正如 3.3.3 节所述，如果操作系统允许应用进程直接使用物理地址，那么会面临两个棘手的问题：第一，难以保证不同应用进程之间的内存隔离性，例如，应用进程 A 在运行过程中可能意外或故意地写应用进程 B 的物理内存，进而导致后者错误运行；第二，难以保证应用进程可用的地址空间是连续且统一的，多个应用进程同时运行时，内存布局的安排会十分复杂。因此，操作系统通过虚拟内存抽象，支持不同应用进程方便且安全地共用物理内存资源。虚拟内存抽象为每个应用进程提供连续且统一的虚拟地址空间，既降低了编程及编译的复杂性，又保证了应用进程仅能访问自己的虚拟地址空间而不能任意访问物理内存。

问题二：应用进程使用虚拟地址，而实际访问物理内存需要通过物理地址，那么虚拟地址是如何转换成物理地址的呢？虚实地址转换（地址翻译）是由 CPU（硬件）和操作系统（软件）配合完成的。一方面，CPU 根据地址翻译规则，将每一条访存指令中的虚拟地址翻译成物理地址，然后再通过总线发送给物理内存进行读写操作。需要强调的是，地址翻译过程对于应用来说是透明的，即应用在编写与编译时都无须考虑地址翻译。另一方面，操作系统负责为每个应用配置虚拟地址到物理地址的翻译规则，但不参与具体的翻译过程。操作系统为不同的应用配置不同的地址翻译规则，从而为它们提供独立的虚拟地址空间。地址翻译体现了“策略与机制分离”的设计思路：操作系统仅负责配置页表（策略），地址翻译则由 CPU 完成（机制），两者通过一个特定的数据结构（页表——记录地址如何映射）实现协同。

在为应用进程提供虚拟内存抽象的任务中，CPU 和操作系统都扮演着重要角色，因此本章包含以下内容：首先介绍 CPU 如何实现从虚拟地址到物理地址的翻译，其次介绍操作系统如何管理虚拟地址与物理地址的映射，最后以 ChCore 为例介绍页表初始化与虚拟内存管理的一种具体实现。

4.1 CPU 的职责：内存地址翻译

本节主要知识点

- 什么是地址翻译？什么是地址翻译规则？
- 为什么需要多级页表？AArch64 体系结构下的多级页表是什么样的？
- 什么是内存的访问异常？哪些情况下会触发访问异常？
- 为什么需要 TLB？何时需要刷新 TLB？

4.1.1 地址翻译

在逻辑上，操作系统把物理内存看成一个大数组，其中每个字节都可以通过与之唯

一对应的地址进行访问，这个地址就是内存的物理地址。CPU 通过总线发送访问物理地址的请求，物理内存中的控制器负责响应请求，使 CPU 能够从物理内存中读取数据或者向其中写入数据，如图 4.1 所示。

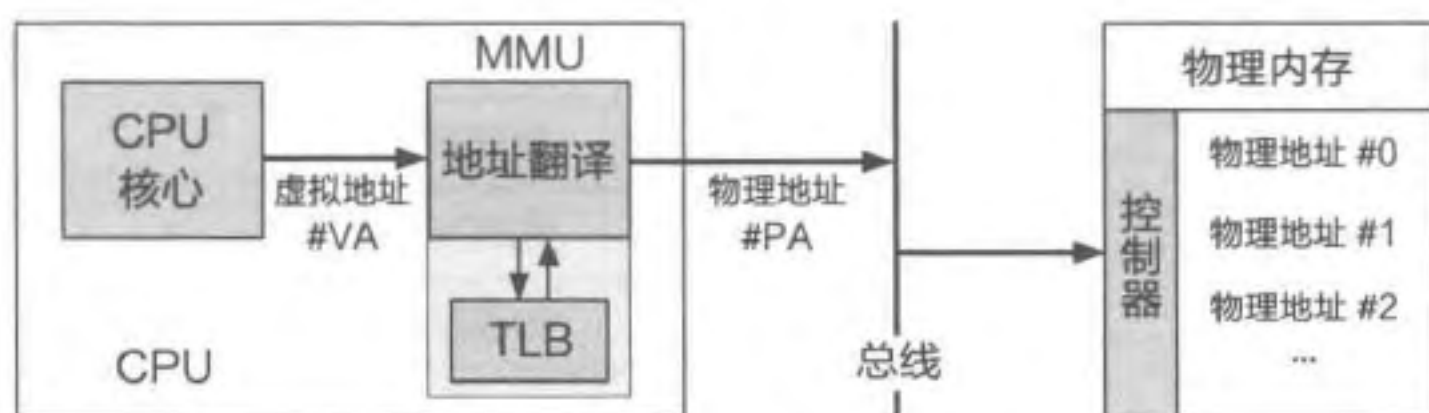


图 4.1 CPU 地址翻译示意图

在引入虚拟内存的抽象后，应用进程使用虚拟地址进行数据读写和指令执行。在应用进程运行过程中，CPU 会将其使用的虚拟地址转换成物理地址，然后通过后者访问物理内存中的数据和代码。虚拟地址转换成物理地址的过程被称为地址翻译。从硬件层面来看，CPU 中的重要部件——内存管理单元（Memory Management Unit, MMU）负责虚拟地址到物理地址的转换。如图 4.1 所示，应用进程在 CPU 核心上运行期间，使用的虚拟地址会由 MMU 进行翻译。当需要访问物理内存时，MMU 翻译出来的物理地址将会通过总线传到物理内存，从而完成物理内存读写请求。此外，现代 CPU 中常包含转址旁路缓存（Translation Lookaside Buffer, TLB）作为加速地址翻译的部件（TLB 是 MMU 内部的硬件单元）。接下来的章节会进一步介绍 MMU 地址翻译和 TLB 的工作原理。

4.1.2 分页机制

MMU 硬件可以支持不同的地址翻译机制，本节重点介绍最常见的分页机制。分页机制的基本思想是将应用进程的虚拟地址空间划分成连续的、等长的虚拟页，同时物理内存也被划分成连续的、等长的物理页。假设页面大小是 4KB，那么虚拟地址 0 ~ 4KB 即为虚拟页 0、虚拟地址 4KB ~ 8KB 即为虚拟页 1；类似地，物理地址 0 ~ 4KB 和 4KB ~ 8KB 分别对应物理页 0 和物理页 1。虚拟页和物理页的页长固定且相等，因而操作系统能够很方便地为每个应用进程构造一张记录从虚拟页到物理页的映射关系表，即页表。为了使 MMU 硬件能够找到页表，需要将页表的起始地址存放在 CPU 的特殊寄存器中，这个寄存器就是页表基地址寄存器，在 AArch64 平台中称为 TTBR（Translation Table Base Register）。

在页表机制下，每个虚拟地址由两部分组成：第一部分标识虚拟地址的虚拟页号；第二部分标识虚拟地址的页内偏移。在地址翻译过程中，MMU 首先解析得到虚拟地址中的虚拟页号，并通过虚拟页号去该应用进程的页表中找到对应条目，然后取出条目中存储的物理页号，最后用该物理页号对应的物理页起始地址加上虚拟地址的页内偏移，得到最终的物理地址。

以图 4.2 中应用 A 的虚拟地址 0x1008 为例（假设每个虚拟页大小为 4KB，即 2^{12} ），

该地址的虚拟页号是 0x1 ($0x1008/2^{12}$), 页内偏移是 0x8 ($0x1008 \% 2^{12}$)。根据应用 A 的页表可知, 虚拟页号 1 对应物理页号 3, 因此虚拟地址 0x1008 对应的物理地址是 0x3008 ($0x3000 + 0x8$)。

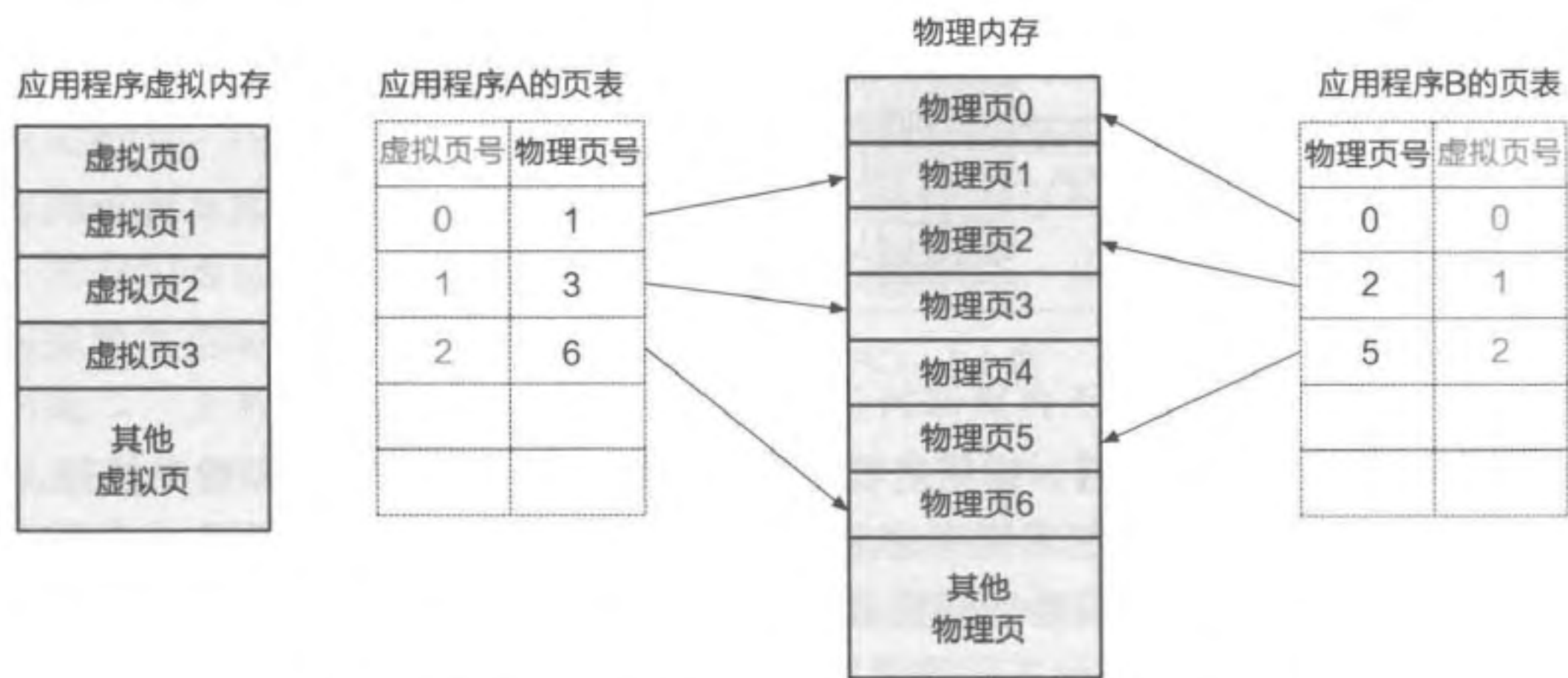


图 4.2 分页机制下的地址翻译规则, 图中所示页表为单级页表

在分页机制下, 操作系统为不同应用进程分配不同的页表, 并且在页表中设置虚拟地址到物理地址的映射关系, 从而实现把应用进程虚拟地址空间中的任意虚拟页映射到物理内存中的任意物理页 (如图 4.2 所示, 物理内存以页为粒度划分给不同的应用)。这既能实现物理内存资源的离散分配, 使应用程序能够使用连续的虚拟地址空间而不用依赖物理地址的连续性, 又能通过在不同页表中映射不同的物理页, 来保证不同应用程序使用的内存互不干扰。当操作系统切换应用程序时, 通过切换页表基地址寄存器中存储的页表即可完成不同进程的虚拟地址空间切换。

小知识：分段机制

在 32 位和 64 位 x86 平台上, 分段机制是 MMU 支持的另一种地址翻译机制。在分段机制下, 操作系统以“段”(一段连续的物理内存)的形式管理 / 分配物理内存。应用进程的虚拟地址空间由若干个不同大小的段组成, 比如代码段、数据段等。当 CPU 访问虚拟地址空间中的某一个段时, MMU 会通过查询段表得到该段对应的物理内存区域 (如图 4.3 所示)。具体来说, 虚拟地址由两部分构成: 第一部分是段号, 标识该虚拟地址属于整个虚拟地址空间中的哪一个段; 第二部分是段内地址, 或称段内偏移, 即相对于该段起始地址的偏移量。段表存储着虚拟地址空间中每一个分段的信息, 其中包括起始地址 (对应物理内存中段的起始物理地址) 和段长。在翻译虚拟地址的过程中, MMU 首先通过段表基址寄存器找到段表的位置, 结合待翻译虚拟地址中的段号, 在段表中定位到对应段的信息 (步骤一); 然后取出该段的起始地址 (物理地址), 加上

待翻译虚拟地址中的段内地址（偏移量），就能够得到最终的物理地址（步骤二）。段表中还存有诸如段长（可用于检查虚拟地址是否超出合法范围）等信息。

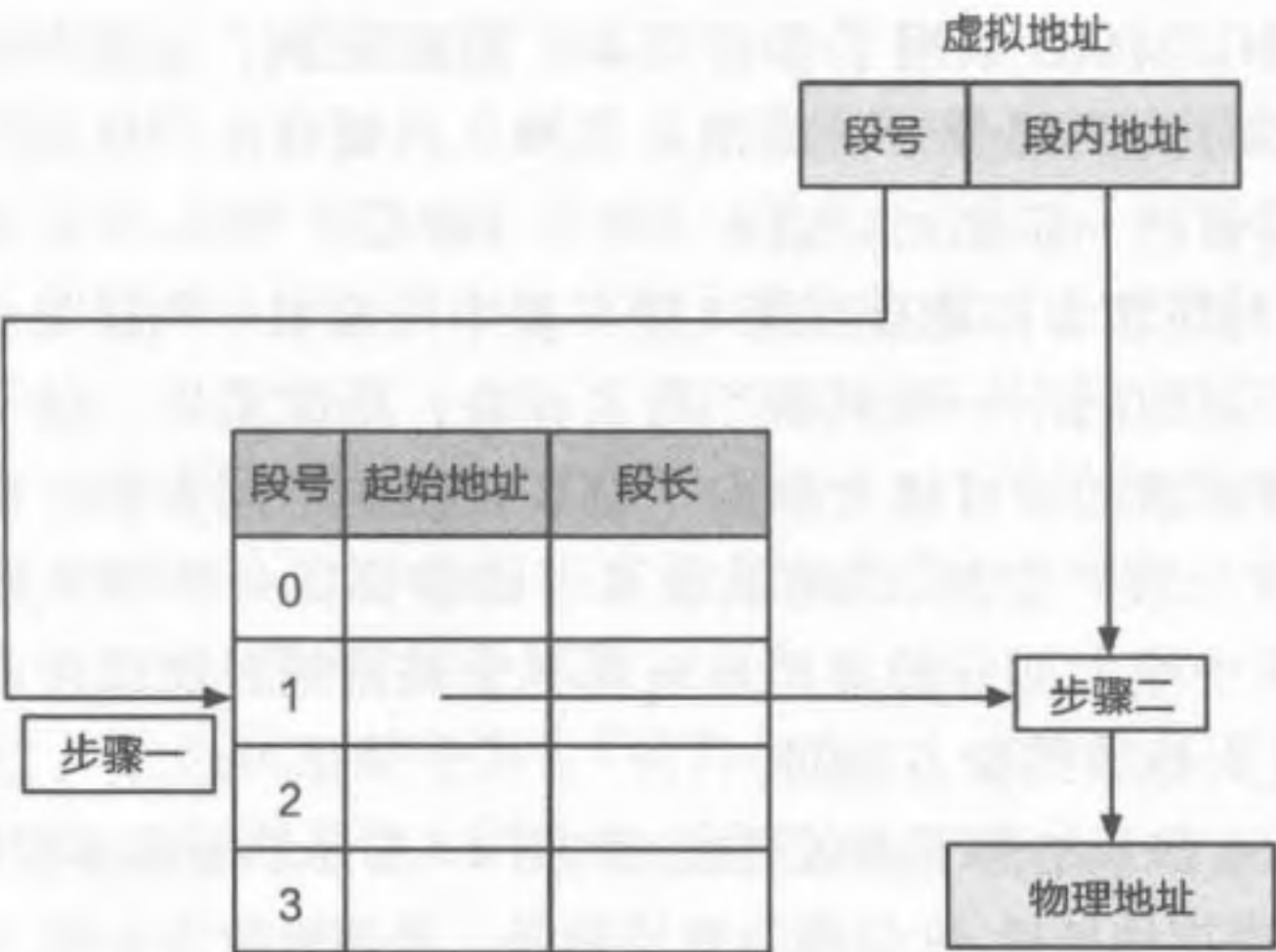


图 4.3 分段机制下地址翻译规则示意图

在分段机制下，不仅虚拟内存空间被划分成不同的段，物理内存也以段为单位进行分配。在虚拟地址空间中，相邻的段所对应的物理内存中的段可以不相邻，因此，操作系统能够实现物理内存资源的离散分配。但是，这种段式分配方式容易导致物理内存中出现外部碎片，即在段与段之间留下碎片空间（不足以映射给虚拟地址空间中的段），从而造成物理内存资源利用率的降低。例如，假设共有 6GB 的物理内存，被划分成 4 段进行分配，第一段为 0 ~ 2GB，第二段为 2GB ~ 3GB，第三段为 3GB ~ 5GB，第四段为 5GB ~ 6GB。如果第二段和第四段被释放，然后又需要分配一个 2GB 的段，虽然此时空闲的物理内存总量为 2GB，但因为这 2GB 内存不连续，分配还是会失败。

Intel 公司在 8086 处理器上开始引入分段机制，然后在 80286 处理器上使用分段机制支持虚拟内存。在后期的处理器上，尤其是在 x86-64 架构之后，基于分页机制的虚拟内存成为主流（不过出于向前兼容的考虑，硬件仍然支持分段机制）^[1]。

练习

4.1 页表基地址寄存器中保存的“页表基地址”是物理地址还是虚拟地址？

4.1.3 多级页表

页表是分页机制中的关键数据结构，负责记录虚拟页到物理页的映射关系。请做一个简单的计算：如果按照图 4.2，使用一张简单的单级页表来记录映射关系，那么对

于 48 位的虚拟地址空间，这个页表有多大？假设页面大小为 4KB，页表中的每一项大小为 8 字节（主要用于存储物理地址），那么一张页表的大小就是 $2^{48}/4K \times 8$ 字节，即 512GB！而且，这张页表占据的物理内存还必须连续！

为压缩页表大小，MMU 采用了多级页表。前面提到，在使用简单的单级页表时，一个虚拟地址将被划分为两部分——虚拟页号和页内偏移。当使用 k 级页表时，一个虚拟地址的虚拟页号将被进一步地划分成 k 个部分（虚拟页号 0， $\dots$ ，虚拟页号 i ， $0 \leq i < k$ ），其中虚拟页号 i 对应该虚拟地址在第 i 级页表中的索引。当任意一级页表中的某一个条目为空时，该条目对应的下一级页表不需要存在，依次类推，接下来的页表同样不需要存在。因此，多级页表的设计极大减少了页表占用的空间大小。换句话说，多级页表允许整个页表结构中出现“空洞”，而单级页表则需要每一项都实际存在。通常，应用进程的虚拟地址空间中绝大部分的虚拟地址都不会被使用，所以多级页表通常具有很多“空洞”，从而能够极大地节约所占空间。

本节结合 AArch64 体系结构下多级页表（如图 4.4 所示）的具体实现进行讲解，采用常见的相关硬件设置：虚拟地址低 48 位参与地址翻译，页表级数为 4 级，虚拟页大小为 4KB。

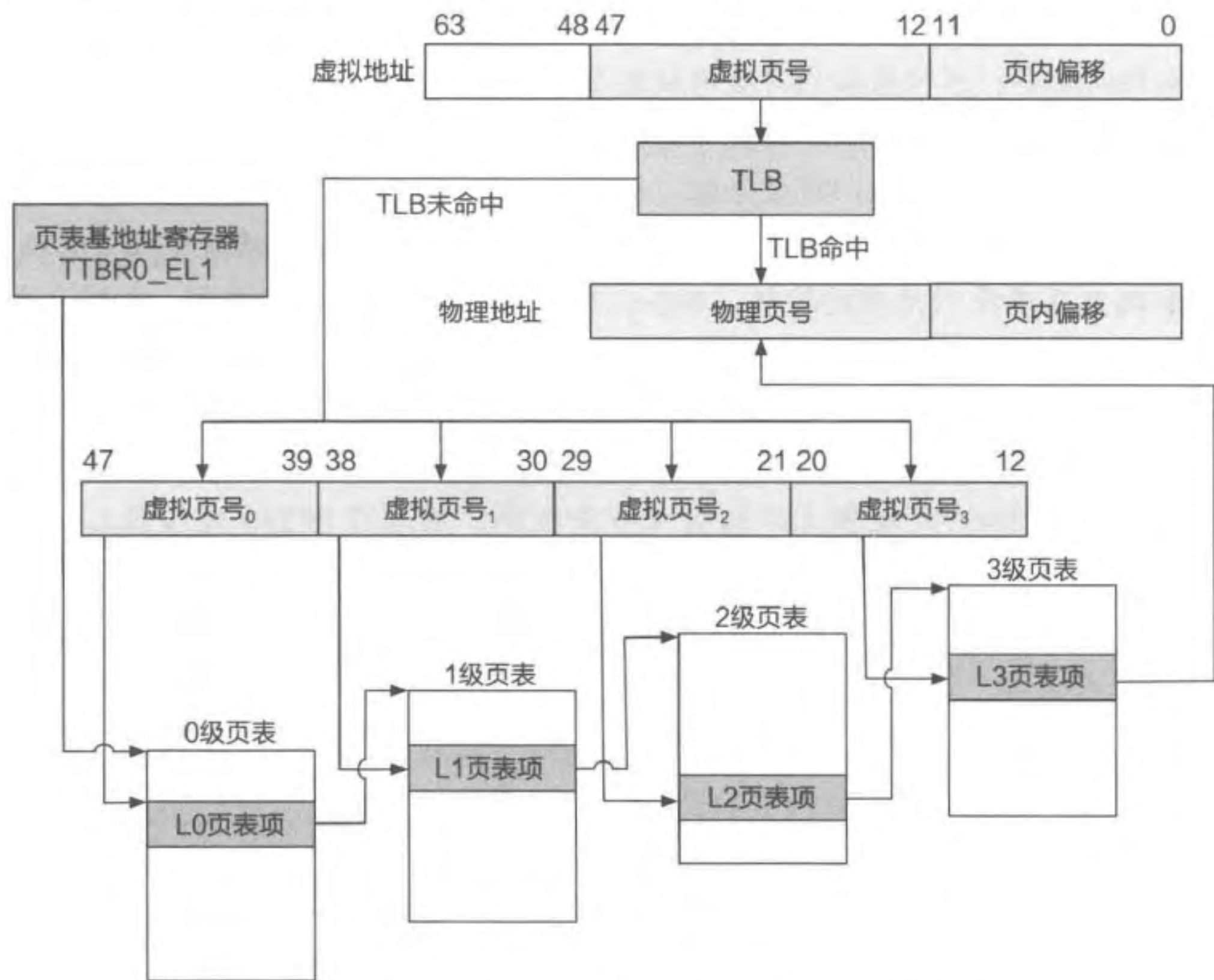


图 4.4 AArch64 体系结构下基于 4 级页表的地址翻译

在此设置下，物理内存被划分为连续的、4KB 大小的物理页，一个虚拟页可以映射并对应一个物理页。正因为页面大小是 4KB，所以虚拟地址的低 12 位 ($2^{12} = 4K$) 对应页内

偏移。整个页表的起始地址（物理地址）存储在页表基地址寄存器中。MMU 硬件在 EL1 特权级（操作系统运行的特权级）中提供了两个页表基地址寄存器，分别是 TTBR0_EL1 和 TTBR1_EL1。当虚拟地址第 63 ~ 48 位全为 0 时，MMU 硬件基于 TTBR0_EL1 寄存器存储的页表进行地址翻译；当虚拟地址第 63 ~ 48 位全为 1 时，MMU 硬件基于 TTBR1_EL1 寄存器存储的页表进行地址翻译。不过，无论使用哪一个页表基地址寄存器，页表结构及地址翻译过程都是相同的。第 0 级（顶级）页表有且仅有一个页表页，页表基地址寄存器存储的就是该页的物理地址。其余每一级页表拥有若干个离散的页表页，每一个页表页也占用物理内存中的一个物理页（4KB）。每个页表项占用 8 字节，用于存储物理地址和相应的访问权限，故一个页表页包含 512 ($4K/8 = 512$) 个页表项。由于 512 项对应 9 位 ($2^9 = 512$)，因此虚拟地址中对应每一级页表的索引都是 9 位。具体来说，一个 64 位的虚拟地址在逻辑上被划分成如下几个部分：

- 第 63 ~ 48 位，全为 0 或者全为 1（硬件要求）。通常操作系统的选择是，应用进程使用的虚拟地址位都是 0。同时这也意味着应用进程的虚拟地址空间大小可以达到 2^{48} 字节。
- 第 47 ~ 39 位：这 9 位作为该虚拟地址在第 0 级页表中的索引值，对应图 4.4 中的虚拟页号 0_0 。
- 第 38 ~ 30 位：这 9 位作为该虚拟地址在第 1 级页表中的索引值，对应图 4.4 中的虚拟页号 1_0 。
- 第 29 ~ 21 位：这 9 位作为该虚拟地址在第 2 级页表中的索引值，对应图 4.4 中的虚拟页号 2_0 。
- 第 20 ~ 12 位：这 9 位作为该虚拟地址在第 3 级页表中的索引值，对应图 4.4 中的虚拟页号 3_0 。
- 第 11 ~ 0 位：由于页面大小是 4KB，所以低 12 位代表页内偏移。

MMU 翻译某虚拟地址时，首先根据页表基地址寄存器中的物理地址找到第 0 级页表页，然后将虚拟地址的虚拟页号 0_0 （第 47 ~ 39 位）作为页表项索引，读取第 0 级页表页中的相应页表项。该页表项中存储着下一级（第 1 级）页表页的物理地址，MMU 按照类似的方式将虚拟地址的虚拟页号 1_0 （第 38 ~ 30 位）作为页表项索引，继续读取第 1 级页表页中的相应页表项。往下类推，MMU 将在第 3 级页表页中的一个页表项里面找到该虚拟地址对应的物理页号，再结合虚拟地址中的页内偏移即可获得最终的物理地址。

这样的 4 级页表结构允许页表内存在“空洞”，操作系统可以在虚拟地址被应用进程使用之后再分配并填写相应的页表页。我们举一个极端的例子来说明多级页表在内存占用方面的优势。假设整个应用进程的虚拟地址空间中只有两个虚拟页被使用，分别位于最低和最高两个虚拟地址。在使用 4 级页表后，整个页表实际上只需要 1 个第 0 级页表页、2 个第 1 级页表页、2 个第 2 级页表页、2 个第 3 级页表页，合计 7 个页表页（即整个页表中大部分都是“空洞”），仅仅占用 28KB 的物理内存空间，远小于单级页表的大小。

小思考

多级页表一定能够减少页表占用的空间吗？

多级页表能减少页表占用的空间，是建立在“应用进程使用的虚拟地址远小于总的虚拟地址空间”这个假设上，如果假设不成立，那么多级页表有可能会比单级页表占用更多的内存。

例如，1995 年，Intel 在奔腾 Pro 处理器上推出了 PAE (Physical Address Extension) 技术，支持 36 位物理地址（即最大 64GB），但虚拟地址仍然只有 32 位。在物理地址是 36 位的情况下，页表项大小为 64 位，每个 4KB 页只能存放 512（即 2^9 ）项，因此使用 3 级页表（即 $2+9+9+12$ ）。对于像 PS 这样的应用，很可能会用满 4GB（32 位）的虚拟地址空间，此时，页表占用的总空间为：第 0 级页表 4KB（只包含 4 个页表项），第 1 级页表 16KB（ $4\text{KB} \times 4$ ），第 2 级页表 8MB（ $4\text{KB} \times 512 \times 4$ ）。而如果使用单级页表，则只需要 8MB 即可。可以看到，在虚拟地址空间用满的情况下，多级页表最末级的页表和单级页表使用的内存大小是一样的——那么多级页表中的其他级页表就是多出来的。多级页表的优势在于不依赖内存页的连续性，因此能够在应用使用虚拟地址较少的情况下大幅度降低页表占用的内存。

小知识：页表的级数

32 位 CPU 采用 32 位的物理地址和虚拟地址，每个页表项为 4 字节，这样每 4KB 的页可以保存 1 024 个页表项，因此普遍使用 2 级页表（即 $32=10+10+12$ ）。当前，智能手机、个人电脑、服务器等平台主要使用 64 位 CPU，一种常见的配置是使用 48 位的物理地址和虚拟地址，此时页表为 4 级（即 $48=9+9+9+9+12$ ）。近年来，x86-64 架构推出了 52 位的物理地址和 57 位的虚拟地址^[2]，操作系统采用 5 级页表以支持更大的虚拟地址和物理地址范围（即 $57=9+9+9+9+9+12$ ）。

练习

- 4.2 为什么单级页表中的每一项都需要存在，且其所在物理内存必须连续？
- 4.3 如果页面大小是 8KB，依然是 4 级页表，当虚拟地址的有效位数分别为 53 位和 48 位时，应该如何划分 64 位虚拟地址呢？
- 4.4 按照 4.1.3 节所述的 4 级页表设置，虚拟地址 0x8080604000 对应的各级页表项索引是什么？虚拟地址 0xFFFF8080604FFF 呢？
- 4.5 按照 4.1.3 节所述的 4 级页表设置：

- (a) 若页表总共占用 16KB，那么至多能够翻译多大的虚拟地址范围？
- (b) 若在页表中填写虚拟地址 0 ~ 16MB 的映射，则页表至少需要占用多少空间？

4.1.4 页表项与大页

如图 4.5 所示，一般来说，多级页表中并非只有最后一级的页表项能够指向物理页，中间级的页表项也能够直接指向物理页，而不是只能指向下一级页表页。当中间级的页表项直接指向物理页时，其指向的是大页，顾名思义，会比下一级页表项指向的物理页的大小更大。此外，页表项除了存储物理页号（下一级页表页或指向的物理页）之外，还会存储一些属性位，允许操作系统设置诸如读写执行等权限。若实际访问所需权限和页表项中设置的权限不一致，则 MMU 会在地址翻译过程中触发访问异常。

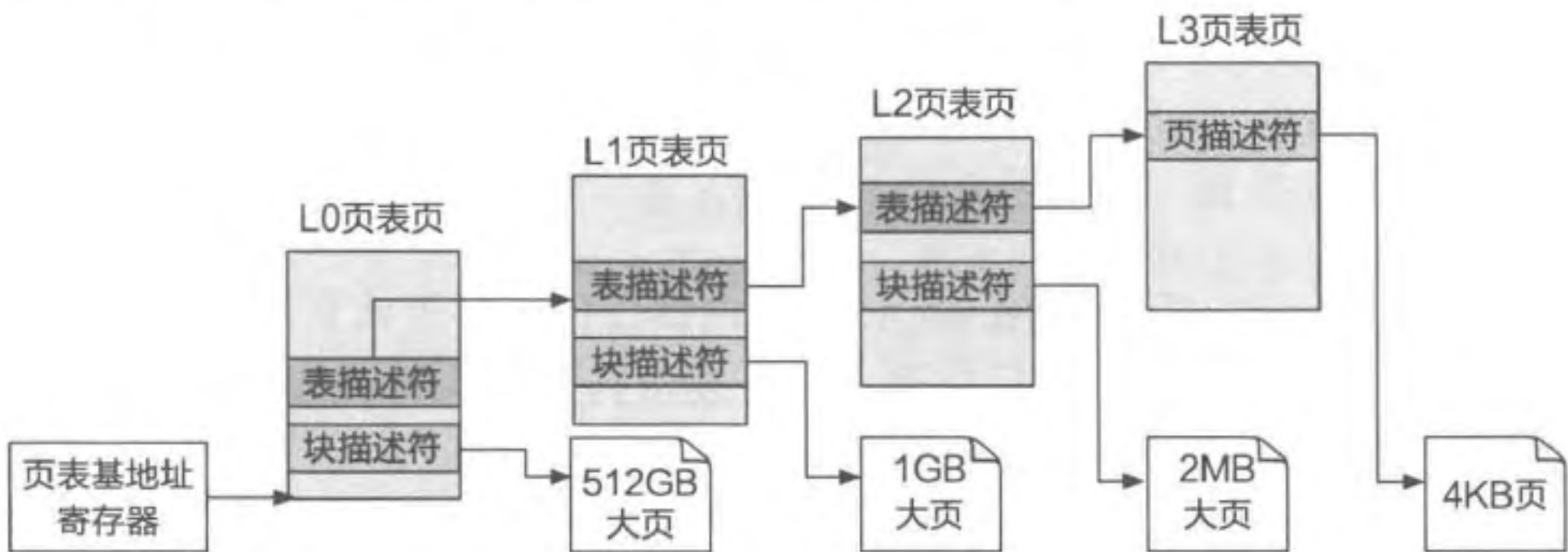


图 4.5 页表项与大页

下面仍然以 AArch64 体系结构的 4 级页表为例进行具体介绍。如图 4.6 所示，每个页表项占用 8 字节（64 位），其中既包括物理页号（PFN），也包括描述页面属性的位。3 级页表项（最后一级页表项）中的 PFN 表示指向的 4KB 物理页，这样的页表项称为页描述符（Page Descriptor）。0 级、1 级、2 级页表项中的 PFN 通常指向下一级页表页，这样的页表项称为表描述符（Table Descriptor）。

除了作为表描述符外，0 级、1 级、2 级页表项还可以作为块描述符（Block Descriptor），其中的 PFN 直接指向 512GB、1GB、2MB 的物理页（大页）。若 0 级、1 级、2 级页表项的第 1 位为 1，则表示该页表项是表描述符，为 0 则表示该页表项是块描述符。有三点需要注意：第一，当使用大页映射时，虚拟大页（即连续的虚拟地址）被映射到相同大小的物理大页（即连续的物理地址），其优点是大大减小了页表，缺点是翻译粒度变粗；第二，大页映射与 4KB 页映射可以在同一个页表中并存；第三，和 4KB 页一样，大页也仅仅是逻辑上的概念，与物理内存并没有直接关系。

页描述符和块描述符的属性位相同，下面介绍本章涉及的页表项中的属性位。

- UXN (Unprivileged eXecute Never): 表示用户态代码对于该页是否具有可执行权限。UXN 为 1 表示不具有可执行权限，为 0 表示具有可执行权限。典型的用法是

利用该位把栈区域标记成不可执行，以避免应用运行过程中由于发生错误或被攻击者攻击而跳转到栈区域执行。



图 4.6 页表项格式

- PXN (Privileged eXecute Never): 表示内核态代码对于该页是否具有可执行权限。PXN 为 1 表示不具有可执行权限，为 0 表示具有可执行权限。典型的用法是利用该位防止内核错误地执行应用进程代码。
- AF (Access Flag): 若 AF 位为 0，则当 MMU 查询到该页表项 (CPU 第一次访问相应虚拟页) 时会触发一种访问异常：访问标志位异常 (Access Flag Fault)。若 AF 位为 1，则不会触发该访问异常。回顾 3.2.2 节，CPU 提供异常向量表，允许操作系统为不同异常设置相应的异常处理函数，当异常发生时，CPU 会立即执行相应的异常处理函数。因此，操作系统可以设置访问异常处理函数，并在该函数中根据硬件提供的错误码判断标志位异常，从而获得页面的访问情况。在后文介绍的操作系统页面回收机制中标志位将会再次出现。
- AP (Access Permission): 用于控制页面的访问权限，可以通过设置不同特权级 (EL0 和 EL1) 实现对页面的不同读写权限，具体见表 4.1。若实际访问所需要的权限违背 AP 表示的访问权限，则当 MMU 查询到该页表项时会触发一种访问异常：访问权限异常 (Access Permission Fault)。
- V (Valid): 有效位，用于表示该页表项是否有效。若 V 位为 0，则当 MMU 查询到该页表项时会触发一种访问异常——缺页异常 (Page Fault)，表示 MMU 在页表中未找到地址翻译所需要的映射。若 V 位为 1，则 MMU 能够完成地址翻译。同

表 4.1 AP 位标识

AP	用户态 EL0	内核态 EL1
00	不可访问	可读可写
01	可读可写	可读可写
10	不可访问	只读
11	只读	只读

样，CPU 允许操作系统为缺页异常设置相应的异常处理函数，在后文介绍的操作系统按需映射机制中缺页异常将会再次出现。注意，在 MMU 查询页表的过程中，只要虚拟地址对应的任一级页表项的 V 位为 0，就会触发缺页异常。

小知识：AF 和 DMB

在 ARMv8.0 中，AF 只能由软件进行设置，即需要操作系统进行设置。在 ARMv8.1-TTMM 体系结构特性下，AF 可以由 MMU 进行设置，即当地址翻译发生时，MMU 会自动将页表项中的 AF 设为 1（不触发访问标志位异常）。此外，页表项中还引入了 DMB 位，表示 Dirty State，即相应的物理页是否被写过。当地址翻译发生且当前执行的是写指令时，MMU 会自动将页表项中的 DMB 位设为 1。

小知识：缺页异常信息

在 x86-64 体系结构下，缺页异常会触发 14 号异常（#PF），并且导致访问出错的虚拟地址会被放在 CR2 寄存器中。在 AArch64 体系结构下，缺页异常并没有专门的异常号，而是与其他一些用户态同步异常共用一个异常号，即 8 号同步异常。操作系统需要根据 ESR（Error Syndrome Register）寄存器中存储的信息来判断发生的异常是不是缺页异常，如果是，则从 FAR_EL1 寄存器中取出发生缺页异常时访问的虚拟地址。类似地，操作系统也是根据 CPU 在 ESR 寄存器中提供的信息判断是不是访问标志位异常等。

练习

- 4.6 为什么上一级页表项能够指向的物理页大小总是下一级页表项能够指向的物理页大小的 512 倍？
- 4.7 当 CPU 在翻译某虚拟地址时，发现相应的最后一级页表项中有效位为 0，且当前是写访问而页表项中 AP 设置的是只读，那么是触发缺页异常还是访问权限异常？

4.1.5 TLB：页表的缓存

多级页表结构能够显著压缩页表的大小，但是会导致地址翻译时长的增加（“时间换空间”的权衡）。具体来说，多级页表结构使得 MMU 在翻译虚拟地址的过程中需要依次查找多个页表页中的页表项，一次地址翻译可能会导致多次物理内存访问。例如，对于图 4.4 所示的 4 级页表，一次地址翻译需要查找 4 级页表项，即需要 4 次访存操作。

为了减少多级页表下地址翻译过程中的访存次数，MMU 引入转址旁路缓存（Translation Lookaside Buffer, TLB）部件来加速地址翻译的过程。具体来说，TLB 缓存了虚拟页号到物理页号的映射关系。我们可以把 TLB 简化成存储着键值对的哈希表，其中键是虚拟页号，值是物理页号。从图 4.4 中可以看到，MMU 会先把虚拟页号作为键去查询 TLB 中的缓存项，若找到则可直接获得对应的物理页号而无须再查询页表。我们称通过 TLB 能够直接完成地址翻译的过程为 TLB 命中（TLB Hit），反之为 TLB 不命中（TLB Miss）。

如图 4.7 所示，一般来说，TLB 硬件采用分层的架构，分为 L1 和 L2 两层。其中，L1 又分为数据 TLB 和指令 TLB，分别用于缓存数据和指令的地址翻译；L2 不区分数据和指令（也存在分离的设计）。作为 CPU 内部的硬件部件，TLB 的体积实际上是极小的，这也就意味着其缓存项的数量是极其有限的。例如，树莓派 4 使用的 AArch64 Cortex-A72 CPU 中，每个 CPU 核心只有约 1 000 条 TLB 缓存项。

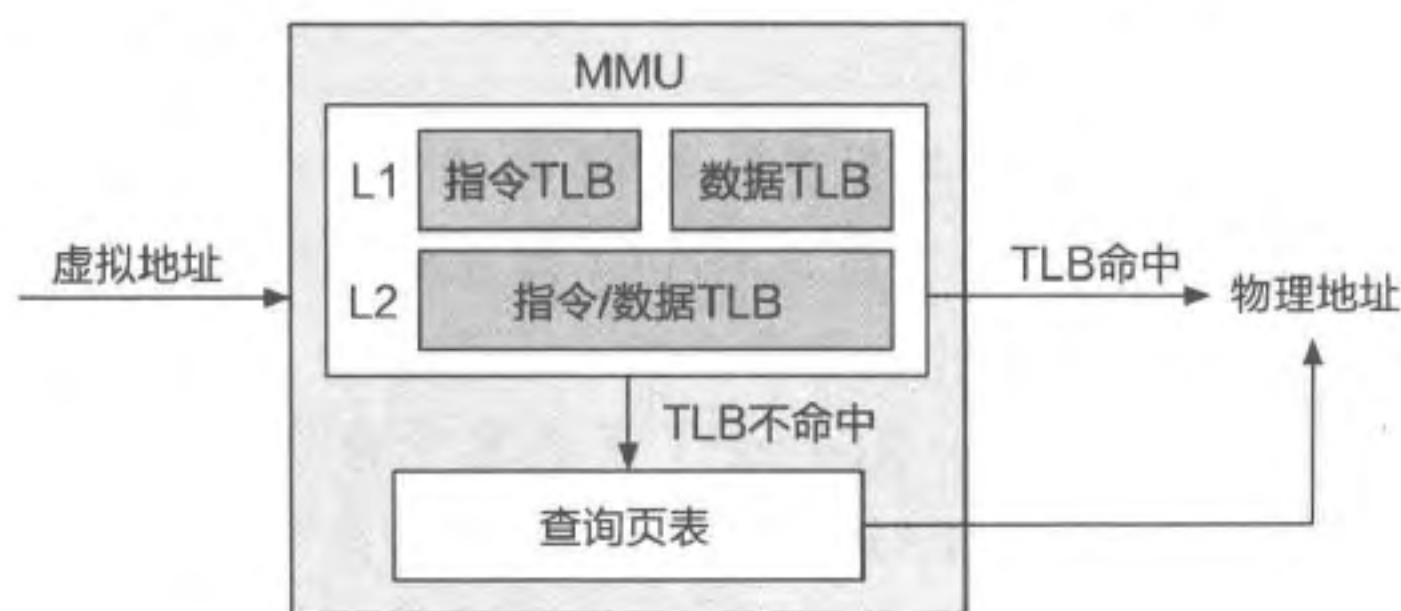


图 4.7 TLB 的结构示意图

由于 TLB 缓存项的数量有限，需要有效地加以利用，才能保证尽量高的 TLB 命中率。在主流的 AArch64 和 x86-64 体系结构下，TLB 在地址翻译过程中是由硬件（MMU）进行管理的。硬件规定了页表基地址的位置以及页表的内部结构，操作系统只需按照硬件的规范来构造和配置页表。当 TLB 不命中时，硬件将通过页表基地址查询页表，找到对应的页表项，并且将翻译结果填写到 TLB 中；若 TLB 已满，则根据硬件预定的策略替换某一项。之后若需要再次翻译同样的虚拟页号，硬件就可以迅速地从 TLB 中直接找到对应的物理页号（TLB 命中）。

小知识：软件管理 TLB

除了由硬件在地址翻译过程中直接管理 TLB 外，有的体系结构设计（如部分 MIPS 和 SPARC 架构的 CPU）允许使用软件在地址翻译过程中对 TLB 进行管理。当发生 TLB 不命中时，硬件会触发异常，然后由操作系统负责根据页表进行地址翻译，使用特殊的指令更新 TLB，并在异常处理完成后恢复 CPU 之前的执行流。不过，操作系统在设计异常处理函数的时候需要格外小心，应避免因 TLB 不命中而导致无限嵌套。这种设计的主要优点是灵活：一方面，操作系统可以使用自定义的页表数据结构；另

一方面，操作系统可以根据运行时的内存访问特点，动态设置 TLB 更新与替换策略。但是，这种设计会导致相应的软件设计变得复杂，同时在性能方面也不一定具有优势。

TLB 的引入带来了一个新的挑战：如何保证 TLB 中的缓存项与当前页表中的映射之间的一致性？请思考这个问题：两个应用进程 A 和 B 使用了同样的虚拟地址 VA，但是对应不同的物理地址 PA1 和 PA2。在应用进程 A 访问 VA 时，TLB 会缓存 VA 到 PA1 的翻译；在切换到应用进程 B 运行后，尽管操作系统更新了 CPU 使用的页表基地址，但是当 B 访问 VA 时，CPU 如果依然从 TLB 中寻找 VA 的翻译，则会导致应用进程 B 的 VA 也翻译成 PA1，进而访问了错误的物理内存地址。导致这个问题的根本原因在于：虽然页表已经发生了变化，但 TLB 没有做相应的更新。由于 TLB 是使用虚拟地址进行查询的，所以操作系统在进行页表切换（应用进程切换）的时候需要主动刷掉 TLB，即 TLB 刷新（TLB Flush）^①。这样，在上面的例子中，当 B 访问 VA 时就会发生 TLB 不命中，然后 CPU 会从 B 的页表中取正确的页表项填充 TLB。

若操作系统在切换应用进程的过程中刷新 TLB，那么每当应用进程被切换到的时候，总是会发生 TLB 不命中的情况，进而不可避免地造成性能损失。那么，有没有一种方法使操作系统能够在切换应用进程时不刷掉 TLB 呢？一种为 TLB 缓存项打上“标签”的设计正是为了避免这样的开销。以 AArch64 体系结构为例，它提供了地址空间标识（Address Space ID, ASID）功能^②。具体来说，操作系统可以为不同的应用进程分配不同的 ASID 作为应用进程虚拟地址空间的标签，并将这个标签写入应用进程页表基地址寄存器中的原空闲位（如 TTBR0_EL1 的高 16 位）。TLB 中的缓存项也会包含 ASID 这个标签，从而使得 TLB 中属于不同应用进程的缓存项可以被区分开。MMU 在翻译地址时，会判断 TLB 缓存项中保存的 ASID 与 TTBR0_EL1 中保存的 ASID 是否一致，只有一致时才会使用该 TLB。因此，在切换页表的过程中，操作系统不再需要清空 TLB 缓存项。

小知识：ASID 数量

ASID 最多有 16 位（ASID 位数由 TCR_EL1 寄存器中的配置信息决定），即同时最多可以有 2^{16} （65 536）个标签。若同时运行的应用进程数量超过 ASID 的最大数量，则操作系统不得不为若干应用进程分配相同的 ASID。

不过在修改页表内容之后，操作系统还是需要主动刷新 TLB 以保证 TLB 缓存和页表项内容一致。AArch64 体系结构提供了多种不同粒度的刷新 TLB 的指令，包括刷新全

① “TLB 刷新”是一种常见的翻译，但更准确地说，“TLB Flush”应该是“刷掉 TLB”，也就是将 TLB 设置为无效。

② 在 x86-64 上，类似 ASID 的功能称为进程上下文标识（Process Context ID, PCID）。

部 TLB、刷新指定 ASID 的 TLB、刷新指定虚拟地址的 TLB 等。操作系统可以根据不同场景选用合适的指令，最小化 TLB 刷新的开销，以获得更好的性能。

练习

- 4.8 为什么硬件仅仅采用简单的 TLB 管理方式，就能够在大多数情况下获得较好的 TLB 命中率？
- 4.9 在 AArch64 平台上：
- (a) 若操作系统的页表基地址存在 TTBR1_EL1 中，应用程序的页表基地址存在 TTBR0_EL1 中，那么在发生系统调用时，是否需要 TLB 刷新？
 - (b) 若操作系统为两个应用程序的页表设置了不同 ASID，那么操作系统在切换进程的过程中是否需要 TLB 刷新？
 - (c) 描述操作系统需要进行 TLB 刷新的两个例子。

4.2 操作系统的职责：管理页表映射

本节主要知识点

- ☐ 操作系统需要为自己配置页表吗？
- ☐ 操作系统如何填写应用进程的页表？
- ☐ 操作系统何时填写应用进程的页表？立即映射与延迟映射有什么区别？
- ☐ 缺页异常与段错误（Segmentation Fault）有什么关系？
- ☐ 如何利用虚拟内存实现内存共享和写时拷贝？
- ☐ 使用大页翻译有什么特点？什么时候适合使用大页？

4.2.1 操作系统为自己配置页表

CPU 在上电启动后会默认使用物理地址，这是因为 MMU 的地址翻译功能还未开启，而操作系统则负责在初始化过程中启用该功能。一旦启用 MMU 地址翻译，CPU 会根据页表对指令执行中涉及的地址进行翻译，即认为这些地址都是虚拟地址，因而操作系统和应用进程在后续运行中都是使用虚拟地址。因此，操作系统除了需要为每个应用进程设置页表外，也需要为自己配置页表。

通常，操作系统为自己配置的页表具有两个特点。第一，操作系统一般使用高虚拟地址，比如对于 AArch64 来说，操作系统使用高 16 位为 1 的虚拟地址；应用进程使用低虚拟地址，在 AArch64 中即为高 16 位为 0 的虚拟地址。第二，操作系统一般会一次性将全部物理内存映射到虚拟地址空间中。具体的映射方式非常简单，即虚拟地址 = 物